

1. Basic-Level Questions

Q1. What is Spring Security and why is it used? - Spring Security is a powerful and highly customizable authentication and access-control framework for Java applications. - It provides protection against common security threats like CSRF, XSS, session fixation, and more.

Q2. Difference between Authentication and Authorization - **Authentication:** Verifying the identity of a user (e.g., username/password). - **Authorization:** Checking if an authenticated user has permission to perform an action.

Q3. Core Components: - `AuthenticationManager` - `UserDetailsService` - `PasswordEncoder` - `SecurityContextHolder`

Q4. Example: In-Memory Authentication

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    @Override
    protected void configure(AuthenticationManagerBuilder auth) throws
Exception {
        auth.inMemoryAuthentication()
            .withUser("admin")
            .password(passwordEncoder().encode("password"))
            .roles("ADMIN");
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

2. Intermediate-Level Questions

Q5. Password Encryption - Spring Security uses `PasswordEncoder` to hash passwords. - Example: `BCryptPasswordEncoder`.

Q6. UserDetails and UserDetailsService - `UserDetails` represents a user. - `UserDetailsService` loads user data from a database or any source.

Q7. Filters and Filter Chain - Spring Security uses a chain of filters. - Example filters: `UsernamePasswordAuthenticationFilter`, `ExceptionTranslationFilter`, `FilterSecurityInterceptor`.

Q8. CSRF Protection

```
http.csrf().disable(); // only if necessary, e.g., for REST APIs
```

Q9. Method-Level Security

```
@EnableGlobalMethodSecurity(prePostEnabled = true)
public class MethodSecurityConfig {}

@PreAuthorize("hasRole('ADMIN')")
public void adminMethod() {}
```

3. Advanced-Level Questions

Q10. JWT Authentication Example

```
public class JwtRequestFilter extends OncePerRequestFilter {
    @Autowired private JwtUtil jwtUtil;
    @Autowired private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
    HttpServletResponse response, FilterChain chain) throws ServletException,
    IOException {
        String authHeader = request.getHeader("Authorization");
        String username = null;
        String jwt = null;

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            jwt = authHeader.substring(7);
            username = jwtUtil.extractUsername(jwt);
        }

        if (username != null &&
        SecurityContextHolder.getContext().getAuthentication() == null) {
            UserDetails userDetails =
            this.userDetailsService.loadUserByUsername(username);
            if (jwtUtil.validateToken(jwt, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new
                UsernamePasswordAuthenticationToken(userDetails, null,
                userDetails.getAuthorities());
```

```

SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}
chain.doFilter(request, response);
}
}

```

Q11. OAuth2 Flows: - Authorization Code - Implicit - Client Credentials - Password Grant

Q12. Stateless vs Stateful Authentication - Stateful: Session stored in server.

- **Stateless:** No server-side session, usually JWT token.

4. Scenario-Based Questions

Q13. Secure endpoint only for ADMIN

```

@PreAuthorize("hasRole('ADMIN')")
@GetMapping("/admin")
public String adminEndpoint() { return "Admin Only"; }

```

Q14. Custom Login Page

```

http.formLogin()
    .loginPage("/custom-login")
    .permitAll();

```

Q15. Security Architecture Diagram Explanation: - **Filter Chain:** Request → Security Filters → AuthenticationManager → SecurityContextHolder → Controller. - **Flow:** 1. Request intercepted by filters. 2. Authentication performed. 3. SecurityContext populated. 4. Access granted/denied based on roles.

5. Best Practices

- Always use `BCryptPasswordEncoder`.
- Keep JWT secret key safe.
- Enable HTTPS.
- Use method-level security for sensitive methods.
- Avoid disabling CSRF unless necessary.

This guide covers theory, practical code examples, and architecture understanding required for interviews.